Project Planning

From Idea to Deployed Product in a Structured Way

■ LEARNING OUTCOME

By the end of this module you will define project scope clearly, choose the right tech stack for any project, create wireframes and user stories, set up a professional project folder structure, and avoid the most common project-killing mistakes.

■ Skills You Will Gain
• Write clear project scope and feature lists that prevent scope creep
• Choose the right tech stack using a decision framework
• Create wireframes and user stories before writing a single line of code
• Set up a professional folder structure for any web project
• Break features into milestones with realistic time estimates
• Define success criteria so you know when the project is done

■ 80% of failed projects fail in the planning phase. Clear scope, right tech stack, and a proper folder structure before coding saves weeks of rework. Professional developers plan first, code second.

SECTION 1

Defining Project Scope

The Problem Statement Formula

A [target user] who wants to [achieve goal] but faces [current problem]. Our solution: a [type of app] that allows them to [core action] so they can [end benefit]. Example: A freelance developer who wants to showcase their work but lacks a professional online presence. Our solution: a personal portfolio site that allows them to display projects with live demos so they can attract clients and employers. Write this BEFORE choosing any technology. Your problem statement drives every tech decision.

SECTION 2

The Project Brief Template

Section	What to Define
Project Name	Short, memorable name for the project
Problem Statement	One paragraph using the formula above
Target Users	Primary user persona: who uses this, how often, on what device
Core Features (MVP)	3-5 MUST-HAVE features for minimum viable product -- everything else is nice-to-have
Out of Scope	Explicitly list what you are NOT building -- prevents feature creep
Tech Stack	Frontend, backend, database, hosting -- with justification for each choice
Success Criteria	How do you define done? Specific, measurable outcomes
Timeline	Milestones with dates: Design, Backend, Frontend, Integration, Testing, Deploy
Risks	What could block you? Missing skills, API limits, time constraints

SECTION 3

Choosing Your Tech Stack

Decision	Options	When to Choose
Frontend Framework	React, Vue, Svelte, Vanilla JS	React: complex UI with lots of state. Vue: gentler learning curve. Vanilla: simple sites, faster load
Backend	Node/Express, Python/FastAPI, Next.js API	Express: flexibility. FastAPI: async Python. Next.js API: same codebase as React frontend
Database	PostgreSQL, MongoDB, SQLite, Supabase	PostgreSQL: relational data. MongoDB: flexible schema. Supabase: managed Postgres with auth

Styling	Tailwind, CSS Modules, styled-components	Tailwind: fastest iteration. Modules: no conflicts. styled-components: JS-driven styles
Auth	JWT, NextAuth, Supabase Auth, Clerk	JWT: full control. NextAuth: Next.js. Supabase/Clerk: managed auth in minutes
Hosting Frontend	Vercel, Netlify, GitHub Pages	Vercel: best for Next.js. Netlify: great for SPAs. GitHub Pages: free for static
Hosting Backend	Railway, Render, Fly.io, AWS	Railway/Render: free tier, easy deploy. Fly.io: global edge. AWS: enterprise scale

SECTION 4

User Stories and Wireframes

```
# USER STORY FORMAT # As a [role], I want to [action], so that [benefit]. Portfolio Site User Stories: As a visitor, I want to see a hero section with the developer's name and role, so that I immediately know who this person is. As a recruiter, I want to filter projects by technology, so that I can quickly find work relevant to my open role. As a potential client, I want to use a contact form, so that I can reach out without exposing my email publicly. CRUD App User Stories: As a user, I want to create a new task with a title, description, and due date, so that I can track what needs to be done. As a user, I want to mark tasks as complete, so that I can see my progress. As a user, I want to delete tasks I no longer need, so that my list stays clean and manageable. ACCEPTANCE CRITERIA (Definition of Done per story): Given [starting context] When [user action] Then [expected result] Example: Given I am on the projects page When I click the 'React' filter button Then only projects tagged with React are visible And the filter button shows an active/selected state
```

SECTION 5

Project Folder Structure

```
# FULL-STACK PROJECT STRUCTURE my-project/ .github/ workflows/ ci.yml # GitHub Actions CI/CD frontend/ # or 'client' public/ # static files served as-is src/ assets/ # images, fonts, icons components/ # reusable UI components ui/ # base components (Button, Card, Input) layout/ # Header, Footer, Sidebar features/ # feature-specific components pages/ # route-level page components hooks/ # custom React hooks context/ # React Context providers services/ # API call functions utils/ # helper functions types/ # TypeScript interfaces vite.config.ts package.json backend/ # or 'server' src/ routes/ # Express route handlers controllers/ # business logic models/ # database models/schemas middleware/ # auth, validation, error handling services/ # external API calls, email etc utils/ # helper functions config/ # environment config server.js package.json .env.example # template for env variables (committed) .env # actual secrets (NEVER committed) .gitignore docker-compose.yml # optional: local dev with DB README.md # setup instructions
```

SECTION 6

Milestone Planning

1	**Week 1 Design** Wireframes for all pages in Figma or on paper. Component inventory. Database schema. API endpoints list.
2	**Week 2 Setup** Git repo, folder structure, environment variables, CI pipeline, database provisioned.
3	**Week 3 Backend** Models, migrations, core API endpoints with basic auth. Postman collection for all endpoints.

4	**Week 4 Frontend** Component library, routing, connect to API, complete CRUD flows working end to end.
5	**Week 5 Polish** Auth flow complete, responsive design, loading/error states, form validation, accessibility.
6	**Week 6 Deploy** Production deployment, domain, SSL, environment variables, smoke test all features live.
7	**Week 7 Launch** README complete, demo video, case study written, shared on GitHub, LinkedIn, portfolio.

■ PRACTICE Q & A

Q1. What is a minimum viable product (MVP)?

> ***A:*** *The smallest version of your product that delivers the core value proposition. Only the features that directly solve the main user problem. Everything else is deferred to v2. Shipping MVP fast teaches you what users actually want.*

Q2. How do you prevent scope creep on a side project?

> ***A:*** *Define 'Out of Scope' explicitly in your brief. For every new idea, ask: does this solve the core user problem? If no, add it to a backlog. Only work on what was in the original scope until the MVP ships.*

Q3. What should a project README include?

> ***A:*** *Project title, one-line description, live demo URL, screenshots/GIF, tech stack, features, local setup steps (clone -> install -> .env -> run), any API keys needed, how to contribute, your contact.*

Q4. How do you estimate how long a feature will take?

> ***A:*** *Break it into the smallest possible tasks. Estimate each task in hours. Double your estimate (everyone underestimates). Add 20% buffer for integration issues. Never estimate in days for tasks smaller than a day.*

Q5. What is the difference between a feature and a user story?

> ***A:*** *A feature is a capability: 'User Authentication'. A user story is from the user's perspective with a why: 'As a returning user, I want to log in with my email, so that my data is private and persistent.' User stories force you to think about value, not just functionality.*

■ Project Planning Cheat Sheet	
Problem Statement	`[User] who wants [goal] but faces [problem]`
MVP	`3-5 MUST-HAVE features only -- ship fast`
Out of Scope	`Explicitly list what you will NOT build`
User Story	`As a [role], I want [action], so that [benefit]`
Acceptance Criteria	`Given/When/Then format per story`
Tech Stack Choice	`Match complexity to project needs`
Wireframe first	`Sketch every page before writing code`
Milestones	`Design -> Setup -> Backend -> Frontend -> Deploy`
Time estimate	`Break to tasks, estimate, then double it`
README	`Demo URL, screenshots, setup steps, tech stack`
.env.example	`Template file for env vars -- commit this`
Success criteria	`Specific measurable done conditions`